Atividade de Revisão

Tema: Base de Dados – Plataforma de eSports “MegaArena”

Contexto:

A startup fictícia MegaArena é uma plataforma nacional de eSports que organiza campeonatos online de jogos como Valorant, CS:GO e League of Legends. A plataforma registra dados de jogadores, partidas, equipes, estatísticas de desempenho e resultados em tempo real. Com o crescimento dos acessos, surgiu a necessidade de adotar soluções escaláveis com banco de dados NoSQL, como o MongoDB, para gerenciar eficientemente os dados, permitir análises e integrar serviços de terceiros como APIs de streaming.

Você foi contratado para auxiliar na modelagem, consulta e otimização dos dados dessa plataforma. Com base nesse cenário, resolva as seguintes questões utilizando o MongoDB:

Objetivo:

Revisar os conteúdos das Aulas 1 a 7 através de um estudo de caso abrangente com aplicação prática em um cenário realista de banco de dados não relacional.

Questões

1. Modelagem com Embedding

Enunciado: Modele um documento que represente um time de eSports com seus jogadores incorporados. O time deve ter nome, modalidade e um array de jogadores contendo nickname, idade e função (função in-game, como suporte, atirador etc.).

```
test> db.times.insertOne({
        nome: "Alfa X Team",
        modalidade: "Valorant",
        jogadores: [
          {nickname: "Ledro", idade: 22, funcao: "Suporte"},
          {nickname: "Killerspf", idade: 42, funcao: "Atirador"},
          {nickname: "MarcinBueno", idade: 21, funcao: "Suporte"},
          {nickname: "DaviSnaider", idade: 20, funcao: "Controlador"}
        ]
      })
```

2. Modelagem com Referencing

Enunciado: Modele um relacionamento 1:N entre coleções jogadores e estatisticas usando referência, onde cada documento de estatísticas representa um jogo com informações como eliminações, mortes e assistências.

```
test> // Criar o jogador individualmente
      const jogadorRes = db.jogadores.insertOne({
      nickname: "HunterX",
      idade: 25,
      funcao: "Atirador",
      time: "Alfa X Team"
      });

      //Criar a estatistica referenciando o ID do jogador acima
      db.estatisticas.insertOne({
        jogador_id: jogadorRes.insertId, //Referencia FK
        partida: "Final Paulista",
        eliminacoes: 25,
        mortes: 4,
        assistencias: 7
      })
```

3. Consulta com Operadores Lógicos

Enunciado: Recupere todos os jogadores que atuam como "suporte" OU "atirador" E têm mais de 18 anos.

```
> db.jogadores.find({
    "idade": { "$gt": 18},
    "$or": [
      { "funcao": "suporte"},
      { "funcao": "atirador"}
    ]
  })
< {
    _id: ObjectId('69cafcce37196f0ac85f9939'),
    nickname: 'BioHazard',
    idade: 19,
    funcao: 'suporte',
    tem_estatistica: true
  }
  {
    _id: ObjectId('69cafcce37196f0ac85f993b'),
    nickname: 'NoobMaster',
    idade: 20,
    funcao: 'suporte',
    tem_estatistica: false
  }
  {
    _id: ObjectId('69cafcce37196f0ac85f993c'),
    nickname: 'ProPlayer',
    idade: 23,
    funcao: 'atirador',
    tem_estatistica: true
  }
test>
```

4. Atualização com Operadores

Enunciado: Adicione o campo status: "ativo" a todos os times que tenham pelo menos 5 jogadores.

```
> db.times.updateMany(
  {"jogadores": { "$size": 5 }},
  {"$set": { "status": "ativo"}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 0,
    modifiedCount: 0,
    upsertedCount: 0
  }
```

5. Exclusão com Condição

Enunciado: Remova todos os jogadores que ainda não participaram de nenhuma partida (sem estatísticas registradas).

```
> db.jogadores.deleteMany({
    "tem_estatistica": false
  })
< {
    acknowledged: true,
    deletedCount: 1
  }
```

6. Agregação e Projeção

Enunciado: Agrupe as estatísticas por jogador e calcule a média de eliminações. Projete somente o nickname e a média calculada.

```
> db.estatisticas.aggregate([
    {
    "$group": {
      "_id": "$nickname",
      "mediaEliminacoes": { "$avg": "$eliminacoes"}
      }
    },
    {
      "$project": {
        "_id": 0,
        "nickname": "$_id",
        "mediaEliminacoes": 1
      }
    }
  ])
< {
    mediaEliminacoes: 10,
    nickname: 'HunterX'
  }
  {
    mediaEliminacoes: 25,
    nickname: null
  }
  {
    mediaEliminacoes: 25,
    nickname: 'Ledro'
  }
```

7. Indexação e Otimização

Enunciado: Crie um índice no campo nickname da coleção jogadores e explique, com consulta, como isso melhora o desempenho em filtros.

```
> db.jogadores.createIndex({ "nickname": 1})
< nickname_1
```

R: Sem o índice, o MongoDB faz um COLLSCAN (Collection Scan), lendo cada documento do disco para achar um nome. Com o índice, ele faz um IXSCAN (Index Scan), indo direto ao endereço físico do dado, o que reduz drasticamente o tempo de resposta e o consumo de CPU.

8. Paginação

Enunciado: Liste os 5 primeiros jogadores de uma equipe ordenados por idade decrescente. Ignore os 3 primeiros resultados.

```
> db.jogadores.find({ "time": "Alfa X Team"})
  .sort({ "idade": -1}) // -1 para decrescente
  .skip(3) // Pula os 3 primeiros
  .limit(5) // Limita s 5 resultados
< {
    _id: ObjectId('69cb013f37196f0ac85f9944'),
    nickname: 'Player_5',
    idade: 21,
    time: 'Alfa X Team'
  }
  {
    _id: ObjectId('69cb013f37196f0ac85f9940'),
    nickname: 'Player_1',
    idade: 20,
    time: 'Alfa X Team'
  }
  {
    _id: ObjectId('69cb013f37196f0ac85f9942'),
    nickname: 'Player_3',
    idade: 19,
    time: 'Alfa X Team'
  }
```